

Programação para IA

Python Avançado e Manipulação de Dados

Eduardo Paiva

CESFI/UDESC - Aula 3

Balneário Camboriú



- **Revisão da Aula Anterior**
- **Python Avançado para IA**
- **Pandas: Manipulação Eficiente de Dados**
- **Processamento de Arquivos LAS**
- **Limpeza e Transformação de Dados**
- **Laboratorio Pratico Intensivo**

Ao final desta aula, voce sera capaz de:

- Utilizar estruturas de dados avançadas em Python
- Manipular DataFrames com Pandas eficientemente
- Processar arquivos LAS (logs de pocos)
- Aplicar tecnicas de limpeza de dados
- Preparar dados para algoritmos de Machine Learning
- Criar visualizações profissionais com Seaborn

O que vimos:

- Álgebra linear: vetores e matrizes
- Estatística descritiva e distribuições
- Correlações entre propriedades petrofísicas
- Operações básicas com NumPy e Pandas

Conexão com hoje:

- Aplicaremos os conceitos matemáticos
- Trabalharemos com dados reais de poços
- Prepararemos dados para Machine Learning

Por que Python é ideal para IA?

- **Sintaxe simples:** Foco na lógica, não na programação
- **Bibliotecas poderosas:** NumPy, Pandas, Scikit-learn
- **Comunidade ativa:** Suporte e recursos abundantes
- **Versatilidade:** Da análise a produção

Aplicações na Eng. de Petroleo:

- Processamento de logs de poços
- Análise de dados sísmicos
- Modelos preditivos de produção
- Otimização de operações

Para além de listas básicas:

Dictionary (Dicionário)

Armazenar propriedades de poços com chaves nomeadas

List Comprehension

Criar listas de forma concisa e eficiente

Sets (Conjuntos)

Operações com dados únicos (formações geológicas)

Aplicação Prática

Organizaremos dados de múltiplos poços de forma estruturada

```
# Estrutura tradicional (problemática)
poco1_phi = 0.15
poco1_k = 100
poco1_sw = 0.30

# Estrutura com dicionário (organizada)
poco1 = {
    'nome': 'Poco_A1',
    'porosidade': 0.15,
    'permeabilidade': 100,
    'saturacao_agua': 0.30,
    'profundidade': 2500,
    'formacao': 'Arenito_Botucatu' }

# Acesso aos dados
print(f"Porosidade: {poco1['porosidade']}")
print(f"Formacao: {poco1['formacao']}")
```

```
# Forma tradicional (verbose)
porosidades = []
for poco in lista_pocos:
    if poco['porosidade'] > 0.15:
        porosidades.append(poco['porosidade'])

# List comprehension (concisa)
porosidades = [poco['porosidade']
               for poco in lista_pocos
               if poco['porosidade'] > 0.15]

# Classificacao de qualidade
qualidade = ['Boa' if phi > 0.15 else 'Moderada'
             for phi in porosidades]

# Conversao de unidades
perm_darcy = [k/1000 for k in permeabilidades_md]
```

Por que Pandas?

- **DataFrames:** Estrutura tabular intuitiva
- **Performance:** Operações otimizadas em C
- **Flexibilidade:** Múltiplos formatos de dados
- **Integração:** Compatível com NumPy e Matplotlib

Principais funcionalidades:

- Leitura/escrita de arquivos (CSV, Excel, JSON)
- Limpeza e transformação de dados
- Agrupamentos e agregações
- Operações temporais

Criando DataFrames: Dados de Poços

```
import pandas as pd
import numpy as np

# Criar DataFrame a partir de dicionario
dados_pocos = {
    'Poco': ['A1', 'A2', 'A3', 'A4'],
    'Porosidade': [0.15, 0.18, 0.12, 0.20],
    'Permeabilidade': [100, 120, 80, 150],
    'Profundidade': [2500, 2480, 2520, 2460],
    'Formacao': ['Botucatu', 'Botucatu', 'Piramboia', 'Botucatu'] }
df = pd.DataFrame(dados_pocos)
print(df)
# Informacoes basicas
print(df.info())
print(df.describe())
```

```
# Selecao de colunas
porosidades = df['Porosidade']
prop_fisicas = df[['Porosidade', 'Permeabilidade']]

# Selecao de linhas (por indice)
primeiro_poco = df.iloc[0]
primeiros_tres = df.iloc[0:3]

# Selecao condicional
pocos_alta_porosidade = df[df['Porosidade'] > 0.15]
botucatu = df[df['Formacao'] == 'Botucatu']

# Multiplas condicoes
bons_pocos = df[(df['Porosidade'] > 0.15) &
                 (df['Permeabilidade'] > 100)]

print(f"Pocos com boa qualidade: {len(bons_pocos)}")
```

```
# Adicionar novas colunas
df['Qualidade'] = df['Porosidade'].apply(
    lambda x: 'Boa' if x > 0.15 else 'Moderada'
)

# Operacoes matematicas
df['Perm_Darcy'] = df['Permeabilidade'] / 1000
df['Indice_Qualidade'] = df['Porosidade'] * df['Permeabilidade']

# Agrupamentos
grupo_formacao = df.groupby('Formacao').agg({
    'Porosidade': ['mean', 'std'],
    'Permeabilidade': ['mean', 'max']
})

# Ordenacao
df_ordenado = df.sort_values('Porosidade', ascending=False)
```

O que são arquivos LAS?

- **Log ASCII Standard:** Formato padrão para logs de poços
- **Estrutura:** Cabecalho + dados tabulares
- **Conteúdo:** Propriedades vs. profundidade
- **Universalidade:** Usado mundialmente

Seções principais:

- ~V (Version): Informações da versão
- ~W (Well): Dados do poço
- ~C (Curve): Descrição das curvas
- ~A (ASCII): Dados numéricos

Estrutura de um Arquivo LAS

~Version Information

VERS. 2.0 : CWLS log ASCII Standard -VERSION 2.0
WRAP. NO : One line per depth step

~Well Information

STRT.M 2000.0 : First depth
STOP.M 2100.0 : Last depth
STEP.M 0.5 : Depth increment
WELL. POCO_A1 : Well name
COMP. PETROBRAS : Company

~Curve Information

DEPT.M : Depth
CALI.IN : Caliper
GR.API : Gamma Ray
NPHI.V/V : Neutron Porosity
RHOB.G/C3 : Bulk Density

~ASCII Data

2000.0	8.5	45.2	0.15	2.35
2000.5	8.7	48.1	0.16	2.33
2001.0	8.9	52.3	0.18	2.31

```
# Instalacao (no terminal)
# pip install lasio
import lasio
import pandas as pd
# Carregar arquivo LAS
las = lasio.read('poco_exemplo.las')
# Informacoes do poco
print(f"Poco: {las.well.WELL.value}")
print(f"Empresa: {las.well.COMP.value}")
print(f"Profundidade: {las.well.STRT.value} - {las.well.STOP.value}m")
# Converter para DataFrame
df_las = las.df()
print(df_las.head())
# Curvas disponiveis
print("Curvas:", las.keys())
```

```
# Estatísticas básicas
print(df_las.describe())
# Limpeza de dados
# Remover valores ausentes (geralmente -999.25)
df_clean = df_las.replace(-999.25, np.nan)
df_clean = df_clean.dropna()
# Calculos petrofísicos
# Porosidade efetiva (exemplo simplificado)
if 'NPFI' in df_clean.columns and 'RHOB' in df_clean.columns:
    df_clean['PHIE'] = (df_clean['NPFI'] +
                        (2.65 - df_clean['RHOB']) / 1.65) / 2
# Identificar zonas de interesse
zona_interesse = df_clean[
    (df_clean['PHIE'] > 0.12) &
    (df_clean['GR'] < 60) ]
print(f"Intervalo produtivo: {len(zona_interesse)} pontos")
```

```
import matplotlib.pyplot as plt
# Configurar subplots
fig, axes = plt.subplots(1, 4, figsize=(15, 10), sharey=True)
depth = df_clean.index # Profundidade (eixo Y invertido)
# Track 1: Caliper e Gamma Ray
axes[0].plot(df_clean['CALI'], depth, 'b-', label='Caliper')
axes[0].set_xlabel('Caliper (in)')
axes[0].invert_yaxis()
axes[0].grid(True)
ax0_twin = axes[0].twinx()
ax0_twin.plot(df_clean['GR'], depth, 'g-', label='GR')
ax0_twin.set_xlabel('Gamma Ray (API)')
# Track 2: Densidade e Neutroao
axes[1].plot(df_clean['RHOB'], depth, 'r-', label='Densidade')
axes[1].plot(df_clean['NPHI'], depth, 'b--', label='Neutroao')
axes[1].set_xlabel('Densidade/Neutroao')
axes[1].legend()
```

Problemas comuns em dados petrolíferos:

- **Valores ausentes:** Falhas de sensores (-999.25)
- **Outliers:** Medições espúrias
- **Unidades inconsistentes:** Diferentes padrões
- **Ruido:** Variações instrumentais

Estratégias de limpeza:

- Identificação de padrões de valores ausentes
- Filtros estatísticos para outliers
- Normalização e padronização
- Interpolação criteriosa

```
# Identificar valores ausentes
print("Valores ausentes por coluna:")
print(df.isnull().sum())
# Valores especificos como ausentes (LAS)
df_clean = df.replace(-999.25, np.nan)
# Estrategias de tratamento
# 1. Remocao (quando poucos dados)
df_sem_na = df_clean.dropna()
# 2. Preenchimento com media
df_clean['GR'].fillna(df_clean['GR'].mean(), inplace=True)
# 3. Interpolacao linear (para logs continuos)
df_clean['RHOB'] = df_clean['RHOB'].interpolate(method='linear')
# 4. Forward fill (carregar ultimo valor valido)
df_clean['CALI'] = df_clean['CALI'].fillna(method='ffill')
# Verificar resultado
print("Apos limpeza:", df_clean.isnull().sum())
```

```
# Metodo IQR (Interquartile Range)
def remove_outliers_iqr(df, column):
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    # Filtrar outliers
    return df[(df[column] >= lower_bound) &
              (df[column] <= upper_bound)]

df_sem_outliers = remove_outliers_iqr(df_clean, 'NPHI')

# Metodo Z-score
from scipy import stats
z_scores = np.abs(stats.zscore(df_clean['GR']))
df_zscore = df_clean[z_scores < 3] # Manter apenas |z| < 3
print(f"Dados originais: {len(df_clean)}")
print(f"Apos remocao outliers: {len(df_sem_outliers)}")
```

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler
# Padronizacao (Z-score normalization)
scaler_std = StandardScaler()
df_padronizado = pd.DataFrame(
    scaler_std.fit_transform(df_clean[['GR', 'NPHI', 'RHOB']]),
    columns=['GR_std', 'NPHI_std', 'RHOB_std'] )
# Normalizacao Min-Max (0 a 1)
scaler_minmax = MinMaxScaler()
df_normalizado = pd.DataFrame(
    scaler_minmax.fit_transform(df_clean[['GR', 'NPHI', 'RHOB']]),
    columns=['GR_norm', 'NPHI_norm', 'RHOB_norm'] )
# Transformacao logaritmica (para permeabilidade)
df_clean['PERM_log'] = np.log10(df_clean['PERM'] + 1)
print("Dados transformados:")
print(df_padronizado.describe())
```

```
# Criar features derivadas
# 1. Razoes petrofisicas
df_clean['PhotoPE_Ratio'] = df_clean['PE'] / df_clean['RHOB']
df_clean['GR_Norm'] = df_clean['GR'] / df_clean['GR'].max()
# 2. Medias moveis (suavizacao)
df_clean['GR_MA'] = df_clean['GR'].rolling(window=5).mean()
# 3. Gradientes (mudancas)
df_clean['RHOB_grad'] = df_clean['RHOB'].diff()
# 4. Classificacoes categoricas
def classify_lithology(gr):
    if gr < 50:
        return 'Arenito'
    elif gr < 100:
        return 'Arenito_Agiloso'
    else:
        return 'Folhelho'
df_clean['Litologia'] = df_clean['GR'].apply(classify_lithology)
```

```
# 5. Binning (discretizacao)
df_clean['Poros_Bins'] = pd.cut(df_clean['NPHI'],
                                bins=[0, 0.10, 0.15, 0.25, 1.0],
                                labels=['Baixa', 'Moderada', 'Boa', 'Excelente'])
```

```
# Agrupar por formacao geologica
```

```
formacao_stats = df_clean.groupby('Litologia').agg({  
    'NPHI': ['mean', 'std', 'min', 'max'],  
    'RHOB': ['mean', 'std'],  
    'GR': ['mean', 'count'],  
    'PERM': ['mean', 'median'] } ).round(3)  
print("Estatísticas por litologia:")  
print(formacao_stats)
```

```
# Percentis por grupo
```

```
percentis = df_clean.groupby('Litologia')['NPHI'].quantile([0.25,  
    0.5, 0.75])  
print("\nPercentis de porosidade:")  
print(percentis)
```

```
# Criar sumario executivo
summary = df_clean.groupby('Litologia').apply(
    lambda x: pd.Series({
        'Espessura_m': len(x) * 0.5,  # Assumindo step 0.5m
        'Poros_Media': x['NPHI'].mean(),
        'Qualidade': 'Boa' if x['NPHI'].mean() > 0.15 else 'Moderada'
    }) )
```

```
# Salvar em diferentes formatos
# 1. CSV (mais comum)
df_clean.to_csv('dados_limpos.csv', index=False)
# 2. Excel (múltiplas abas)
with pd.ExcelWriter('relatorio_poco.xlsx') as writer:
    df_clean.to_excel(writer, sheet_name='Dados_Limpos', index=False)
    formacao_stats.to_excel(writer, sheet_name='Estatísticas')
    summary.to_excel(writer, sheet_name='Sumário')
# 3. JSON (para APIs)
df_clean.to_json('dados.json', orient='records')
# 4. Pickle (preserva tipos Python)
df_clean.to_pickle('dados.pkl')
# 5. HDF5 (para datasets grandes)
df_clean.to_hdf('dados.h5', key='df', mode='w')
print("Dados exportados com sucesso!")
```

Exercicio Prático 1: Criação de Dataset

Objetivo: Criar e manipular um dataset sintético de pocos

Tarefa

- 1 Criar DataFrame com 50 pocos
- 2 Incluir: nome, coordenadas, propriedades petrofísicas
- 3 Adicionar valores ausentes propositalmente
- 4 Aplicar técnicas de limpeza
- 5 Calcular estatísticas por formação

Tempo Estimado

15 minutos - Trabalho em duplas

Arquivo base no Google Colab

Exercício Prático 2: Processamento LAS

Objetivo: Processar arquivo LAS real

Tarefa

- 1 Carregar arquivo LAS fornecido
- 2 Explorar estrutura e metadados
- 3 Limpar dados (valores ausentes/outliers)
- 4 Calcular porosidade efetiva
- 5 Identificar zonas produtivas
- 6 Criar visualização de logs

Tempo Estimado

20 minutos - Trabalho individual

Arquivo LAS disponível no Colab

Além do Matplotlib básico:

- **Seaborn:** Gráficos estatísticos elegantes
- **Plotly:** Visualizações interativas
- **Matplotlib:** Controle total e personalização

Tipos de gráficos úteis:

- **Heatmaps:** Correlações entre propriedades
- **Pairplots:** Relações multivariadas
- **Boxplots:** Distribuição por formação
- **Histogramas:** Distribuições de propriedades

```
import seaborn as sns
import matplotlib.pyplot as plt

# Configurar estilo
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (12, 8)

# 1. Heatmap de correlacao
correlacao = df_clean[['GR', 'NPHI', 'RHOB', 'PE']].corr()
plt.figure(figsize=(8, 6))
sns.heatmap(correlacao, annot=True, cmap='coolwarm', center=0)
plt.title('Correlacao entre Propriedades Petrofisicas')
plt.show()

# 2. Pairplot (relacoes multivariadas)
sns.pairplot(df_clean[['GR', 'NPHI', 'RHOB', 'Litologia']],
             hue='Litologia', diag_kind='hist')
plt.show()
```

```
# 3. Boxplot por litologia
plt.figure(figsize=(10, 6))
sns.boxplot(x='Litologia', y='NPHI', data=df_clean)
plt.title('Distribuicao de Porosidade por Litologia')
plt.ylabel('Porosidade Neutroa')
plt.show()
```

Projeto Individual:

- ❶ **Adquirir dados reais:** Baixar arquivo LAS público
 - Fontes: BDEP (ANP), NLOG (Holanda), USGS
- ❷ **Análise completa:**
 - Processamento e limpeza
 - Feature engineering petrofísico
 - Identificação de zonas produtivas
 - Visualizações profissionais
- ❸ **Relatorio tecnico:**
 - Notebook comentado
 - Interpretacao geologica
 - Recomendacoes tecnicas

Entrega: Antes da Próxima aula (via moodle)

Aula 4 - Manipulação e Visualização de Dados:

- Análise exploratoria avançada
- Técnicas de visualização científica
- Dashboards interativos
- Análise estatística aprofundada

Preparação recomendada:

- Revisar conceitos de hoje
- Praticar com dados próprios
- Explorar documentação do Plotly

Continuaremos construindo expertise em dados!

Documentação oficial:

- **Pandas:** pandas.pydata.org/docs/
- **Lasio:** lasio.readthedocs.io/
- **Seaborn:** seaborn.pydata.org/
- **Plotly:** plotly.com/python/

Livros recomendados:

- **McKinney, W.** - Python for Data Analysis
- **VanderPlas, J.** - Python Data Science Handbook
- **Geron, A.** - Hands-On Machine Learning

Datasets publicos:

- BDEP/ANP (Brasil), NLOG (Holanda), USGS (EUA)

Eduardo Paiva

Email: eduardo.paiva@udesc.br

Sala: 38

Atendimento: Quintas-feiras, 13:30 as 15:00h

Proxima aula: Visualizacao avancada!

Obrigado!